Month 2 – Cryptographic Layer & Base Core

Research Report: Cryptographic Hashing, Access Control, and Log-Chaining Fundamentals

1. Introduction

Month 1 of this internship produced the Entity Relationship Diagram (ERD) for an immutable digital chain of custody system, defining six entities — User, Case, Evidence, Custody Record, Audit Log, and Location — and establishing that evidence integrity depends on cryptographic hashing, and that Custody Record and Audit Log entries must remain permanent once created.

Month 2 addresses the cryptographic and architectural foundations required to implement that design: how file integrity hashing works, how database access can be restricted to prevent modification or deletion, and how log entries can be cryptographically linked so that tampering becomes detectable. This report presents the core knowledge underlying each of these three areas.

2. Cryptographic Hashing Fundamentals

A cryptographic hash function takes an input of any size — a file, a string, a block of data — and produces a fixed-length output called a **digest** or **hash value**. Three properties make a hash function suitable for integrity checking:

- **Determinism** — the same input always produces the same output.
- **Avalanche effect** — changing even a single bit of the input produces a completely different, unpredictable output.
- **Collision resistance** — it should be computationally infeasible to find two different inputs that produce the same hash.

Because of these properties, a hash acts as a fingerprint: recomputing a file's hash and comparing it against a previously recorded value reveals whether the file has changed, without needing to compare the file contents directly.

2.1 SHA-256

SHA-256 is part of the SHA-2 family (Secure Hash Algorithm 2), published by NIST. It produces a 256-bit (64 hexadecimal character) digest and remains the current industry standard for integrity verification, digital signatures, and forensic evidence handling. No practical collision has been found against it, and it is fast enough to hash large files in chunks without significant overhead.

2.2 SHA-3

SHA-3 is the newer NIST standard, built on a different internal structure (Keccak's sponge construction) rather than the Merkle–Damgård structure used by SHA-1/SHA-2. It is not a replacement for SHA-256 in most current systems but serves as a structurally independent alternative — useful if a weakness were ever found in the SHA-2 family, since SHA-3 would not share the same underlying design flaw.

2.3 BLAKE2

BLAKE2 is a modern hash function designed to be faster than SHA-2 while offering comparable or stronger security guarantees. It is increasingly used in performance-sensitive integrity-checking contexts.

2.4 Why MD5 and SHA-1 are excluded

MD5 and SHA-1 were once standard but are now considered cryptographically broken: practical collision attacks exist for both, meaning two different files can be crafted to produce the same hash. Neither should be relied on as a primary integrity mechanism in a chain-of-custody context, where legal admissibility depends on the hash being trustworthy.

2.5 Application to file integrity

For evidence files, the hashing process reads the file in binary mode (not text mode, so the process is format-agnostic across documents, images, disk images, and executables) and streams it in fixed-size chunks rather than loading the whole file into memory. The resulting digest, together with the algorithm used, is what populates the `HashValue` and `HashAlgorithm` attributes on the Evidence entity from the Month 1 ERD. Verification later consists of recomputing the hash and confirming it matches the stored value.

3. Restricting Database Access to Prevent Modification or Deletion

A chain of custody system depends on the guarantee that once a record is written, it cannot be silently altered. This is achieved through an **append-only** (also called **Write-Once, Read-Many / WORM**) data access pattern.

3.1 The principle of least privilege

Least privilege means every part of a system is given only the permissions it strictly needs to perform its function, and nothing more. Applied to a chain of custody database, the component responsible for writing Audit Log or Custody Record entries needs the ability to **insert** and **read** data — it never needs to update or delete a record, because those entities are defined as permanent by design.

3.2 Enforcement at two layers

- **Application layer** — the code that interacts with the database is written so that no update or delete operation is exposed. If the capability does not exist in the code, it cannot be invoked by mistake or misuse.
- **Database layer** — as a second, independent safeguard, the database account itself has UPDATE and DELETE privileges revoked at the engine level (via SQL `GRANT`/`REVOKE`). This means that even if the application layer were bypassed or compromised, the database itself would refuse the operation.

Using both layers together is known as **defense in depth** — no single point of failure can compromise the immutability guarantee.

3.3 Relevance to the ERD

This directly implements what the Month 1 report already specified: that Audit Log entries are "appended and never changed after creation." The same principle extends naturally to Custody Record, since every transfer of evidence must remain a permanent, unaltered part of its history.

4. Sequential Cryptographic Log-Chaining

Restricting access prevents *authorized* modification, but a stronger guarantee is needed: proof that no modification occurred at all, even one made outside normal channels. This is provided by chaining log entries together cryptographically.

4.1 The linking mechanism

Each log entry stores the hash of the entry immediately before it, alongside its own data:

entry_hash = SHA256(previous_entry_hash + entry_data + timestamp)

The very first entry in the chain links to a fixed starting value (a "genesis hash"), since no prior entry exists. Every subsequent entry's hash depends on the content of the entry before it.

4.2 Why this makes tampering detectable

If any past entry is altered after the fact — even by changing a single character — its hash no longer matches what the next entry in the chain expects. That mismatch propagates forward through every entry after it. Re-walking the chain and recomputing each hash in sequence will reveal exactly where the break occurred, pinpointing the tampered entry.

This is structurally the same linking principle used in blockchains, though a chain of custody log does not require the additional complexity of distributed consensus — a single authoritative, append-only chain is sufficient.

4.3 Academic grounding

This approach is formally described in Schneier and Kelsey's 1999 work on secure audit logs, which established the model of using cryptographic hash chains to make log tampering detectable rather than merely prohibited. It remains the foundational reference for tamper-evident logging design.

4.4 Relevance to the ERD

Log-chaining upgrades the Audit Log entity from a table that is *policy-protected* against edits to one that is *provably* unaltered — directly supporting the Month 1 report's claim that the design satisfies forensic integrity and aligns with ISO/IEC 27037's requirements for evidence handling.

5. Summary

Concept	Guarantees	ERD relevance
Cryptographic hashing (SHA-256/SHA-3/BLAKE2)	File contents can be verified as unaltered	Populates `Evidence.HashValue` / `HashAlgorithm`
Append-only access layer (least privilege, defense in depth)	Records cannot be modified or deleted through normal use	Enforces immutability of `Audit Log`, `Custody Record`

Sequential hash-chaining	Any unauthorized tampering is detectable, not just forbidden	Strengthens `Audit Log` to a tamper-evident state

Together, these three areas form the cryptographic and architectural foundation on which the chain of custody system's immutability claims rest, as defined in the Month 1 ERD.